DraggableScrollableSheet in Flutter – Complete Documentation

1. Introduction

`DraggableScrollableSheet` is a Flutter widget that creates a bottom sheet which can be drag expand or collapse.

It is commonly used in: - Google Maps style UI - Music player bottom panels - Product deta - Modern mobile app designs

It allows the user to drag the sheet between minimum and maximum sizes.

---

2. Basic Syntax

```
DraggableScrollableSheet(
  initialChildSize: 0.3,
  minChildSize: 0.2,
  maxChildSize: 0.8,
  builder: (context, scrollController) {
    return Container();
  },
)
```

---

3. Important Properties

Property	Description
initialChildSize	Initial height (`0.0` to `1.0` of screen height)
minChildSize	Minimum draggable height
maxChildSize	Maximum draggable height
expand	Whether it fills available space
builder	Builds the content inside sheet

Values are percentages of screen height. Example: `0.5` = `50` % of screen height

---

4 . Simple Working Example

```
import 'package:flutter/material.dart';

class DraggableExample extends StatelessWidget {
  const DraggableExample({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Stack(
        children: [
          Container(
            color: Colors.blueGrey,
            child: const Center(
              child: Text(
                "Background Content",
                style: TextStyle(color: Colors.white, fontSize: 20),
              ),
            ),
          ),

          DraggableScrollableSheet(
            initialChildSize: 0.3,
            minChildSize: 0.2,
            maxChildSize: 0.8,
            builder: (context, scrollController) {
              return Container(
                decoration: const BoxDecoration(
                  color: Colors.white,
                  borderRadius: BorderRadius.vertical(
                    top: Radius.circular(20),
                  ),
                ),
                child: ListView.builder(
                  controller: scrollController,
                  itemCount: 20,
                  itemBuilder: (context, index) {
                    return ListTile(
                      title: Text('Item \$index'),
                    );
                  },
                ),
              );
            },
          ),
```

```
        ],
      ),
    );
  }
}
```

---

5 . Why ScrollController is Important

Inside the builder:

```
builder: (context, scrollController)
```

You MUST attach `scrollController` to your scrollable widget:

```
ListView(
  controller: scrollController,
)
```

If you don't use it, the dragging will not work properly.

---

6 . With SingleChildScrollView Example

```
DraggableScrollableSheet(
  initialChildSize: 0.4,
  minChildSize: 0.3,
  maxChildSize: 0.9,
  builder: (context, scrollController) {
    return Container(
      color: Colors.white,
      child: SingleChildScrollView(
        controller: scrollController,
        child: Column(
          children: List.generate(
            30,
            (index) => ListTile(title: Text('Data \$index')),
          ),
        ),
      ),
    );
```

```
  },
)
```

---

7. Common Use Cases

1. Map + bottom location details
2. Music player expandable panel
3. Shopping product preview
4. News preview panel
5. Chat contact list preview

---

8. Best Practices

- Always wrap inside Stack if you want background content
- Use rounded top border for modern UI
- Use ListView instead of Column for long lists
- Keep maxChildSize below 1.0 for better UX
- Avoid heavy UI inside builder

---

9. Common Mistakes

❌ Forgetting to use scrollController ❌ Using Column without scroll ❌ Setting minChildSize great
initialChildSize ❌ Using it inside another scrollable incorrectly

---

10. Advanced: Snap Behavior (Flutter 3.7+)

You can enable snap effect:

```
DraggableScrollableSheet(
  snap: true,
  snapSizes: [0.3, 0.6, 0.9],
)
```

This allows the sheet to snap to defined sizes.

---

11. Folder Structure Example

```
lib/
 ├── presentation/
 │    ├── screens/
 │    │    └── draggable_screen.dart
 │    └── widgets/
 │         └── custom_bottom_sheet.dart
```

---

12. Summary

DraggableScrollableSheet allows you to:

✔️ Create modern expandable bottom sheets ✔️ Build interactive UI ✔️ Combine drag + scroll behavior ✔️ Improve user experience in complex layouts

---

You can extend this documentation with: - Animation customization - Integration with Google Maps Architecture example - Combining with Provider or Bloc - Using with ModalBottomSheet